ISA Specification

WM-8 five-stage pipelined washing-machine microcontroller

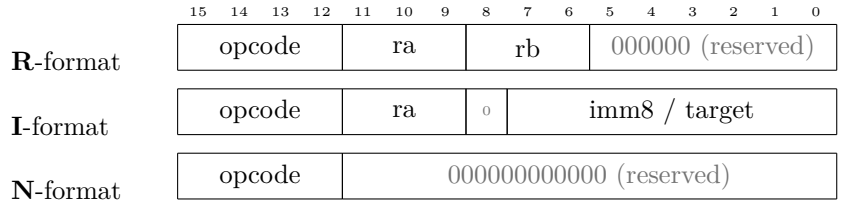
ELL7282/ELL782 Introduction to Computer Architecture – Assignment 2

1 Programmer's model

- 8-bit data path; all arithmetic is modulo 256. Eight general registers `R0`–`R7`; none is hard-wired to zero and none is cleared by reset (software initialises every register it reads).
- 16-bit fixed-length instructions in a 256-word instruction ROM. The 8-bit `PC` addresses instruction *words*, is 0 after reset and normally increments by one (wrapping `0xFF`→`0x00`).
- Data space: 256 byte addresses – RAM `0x00`–`0x7F`, reserved `0x80`–`0xEF` (reads 0, writes ignored), peripherals `0xF0`–`0xFF` (see `memory_map.pdf`). Only register-indirect addressing `[Rs]` is provided.
- No condition flags. Branches test a register for zero; the custom `SGEU` produces a 0/1 compare result.
- Branch/jump targets are *absolute* 8-bit instruction-word addresses; there are no delay slots (the two wrong-path instructions fetched after a taken branch are squashed in hardware).

2 Instruction formats (three formats)

Register fields are always in the same place, so the register file can be read in ID before the instruction is fully decoded: **ra** = `IR[11:9]` (destination / first source / store data / tested register) and **rb** = `IR[8:6]` (second source / address register).



Reserved bits. Software (and the assembler) must write reserved bits as 0. The hardware never decodes them: `IR[5:0]` of R-format, `IR[8]` of I-format, `IR[11:0]` of N-format and the `ra` field of `JMP` are ignored, so any value there produces the same, deterministic behaviour. Because the `rb` field physically overlaps `imm8[7:6]` and `IR[8]`, the hardware only treats a field as a register source when the opcode actually reads it (*source-use* bits `UA`/`UB`, Section 5); otherwise immediates would create false load-use stalls.

3 Opcode map and semantics

Op	Mnemonic	Fmt	Semantics	Reads	Writes
`0x0`	`NOP`	N	no state change, `PC`←`PC`+1	–	–
`0x1`	`LDI Rd, imm8`	I	`Rd`←`imm8` (bit pattern copied, no extension)	–	Rd
`0x2`	`MOV Rd, Rs`	R	`Rd`←`Rs`	rb	Rd
`0x3`	`ADD Rd, Rs`	R	`Rd`←(`Rd`+`Rs`) mod 256	ra, rb	Rd
`0x4`	`SUB Rd, Rs`	R	`Rd`←(`Rd`−`Rs`) mod 256	ra, rb	Rd
`0x5`	`AND Rd, Rs`	R	`Rd`←`Rd` AND `Rs`	ra, rb	Rd
`0x6`	`LD Rd, [Rs]`	R	`Rd`←`DATA[Rs]` (read in MEM)	rb	Rd
`0x7`	`ST Rd, [Rs]`	R	`DATA[Rs]`←`Rd` (written once, in MEM)	ra, rb	memory/IO
`0x8`	`ADDI Rd, imm8`	I	`Rd`←(`Rd`+`imm8`) mod 256 (`0xFF` adds −1)	ra	Rd
`0x9`	`BEQZ Rs, target`	I	if `Rs`=0: `PC`←`target`	ra	PC
`0xA`	`BNEZ Rs, target`	I	if `Rs`≠0: `PC`←`target`	ra	PC
`0xB`	`JMP target`	I	`PC`←`target` (ra field ignored)	–	PC
`0xC`	`SGEU Rd, Rs` **(custom)**	R	`Rd`←(`Rd`$\geq_u$`Rs`) ? `0x01` : `0x00`	ra, rb	Rd
`0xD`	*unassigned*	–	executes exactly as `NOP` (all-zero control word)	–	–
`0xE`	*unassigned*	–	executes exactly as `NOP` (all-zero control word)	–	–
`0xF`	`HALT`	N	older instructions complete, younger ones are squashed, fetch stops until reset	–	halt latch

Unassigned opcodes `0xD`/`0xE` decode to one-hot lines that feed no OR term of the control word, so all of `RW`, `MW`, `BZ`, `BNZ`, `JP` and `HT` are 0: no register, memory, I/O or PC side effect, and they cannot stall the pipeline (`UA`=`UB`=0).

4 Bit-level decode table

Instruction	[15:12]	[11:9]	[8:6] / [8]	[5:0] / [7:0]	unused bits
NOP	0000	000	000	000000	[11:0] reserved
LDI Rd,imm	0001	d	[8]=0	imm8	[8]
MOV Rd,Rs	0010	d	s	000000	[5:0]
ADD Rd,Rs	0011	d	s	000000	[5:0]
SUB Rd,Rs	0100	d	s	000000	[5:0]
AND Rd,Rs	0101	d	s	000000	[5:0]
LD Rd,[Rs]	0110	d	s	000000	[5:0]
ST Rd,[Rs]	0111	d (data)	s (address)	000000	[5:0]
ADDI Rd,imm	1000	d	[8]=0	imm8	[8]
BEQZ Rs,t	1001	s	[8]=0	target	[8]
BNEZ Rs,t	1010	s	[8]=0	target	[8]
JMP t	1011	000	[8]=0	target	[11:8]
SGEU Rd,Rs	1100	d	s	000000	[5:0]
HALT	1111	000	000	000000	[11:0]

Machine-code examples (one per format)

Format	Assembly	Binary (op | ra | rb/imm)	Hex
R	ADD R1, R2	0011 | 001 | 010 | 000000	0x3280
R	ST R1, [R4]	0111 | 001 | 100 | 000000	0x7300
R	SGEU R1, R4	1100 | 001 | 100 | 000000	0xC300
I	LDI R1, 0x64	0001 | 001 | 0 | 01100100	0x1264
I	ADDI R2, -3	1000 | 010 | 0 | 11111101	0x84FD
I	BEQZ R1, 0x0A	1001 | 001 | 0 | 00001010	0x920A
I	JMP 0x10	1011 | 000 | 0 | 00010000	0xB010
N	HALT	1111 | 000000000000	0xF000

5 Control word produced by the decoder

The 4-bit opcode drives a gate-level 4:16 decoder; the 13-bit control word CW is formed by OR-ing one-hot lines and travels in the ID/EX register.

CW bit	RW 0	MR 1	MW 2	UA 3	UB 4	YI 5	AOP[1:0] 7:6	SB 8	BZ 9	BNZ 10	JP 11	HT 12
NOP, 0xD, 0xE	0	0	0	0	0	0	00	0	0	0	0	0
LDI	1	0	0	0	0	1	10	0	0	0	0	0
MOV	1	0	0	0	1	0	10	0	0	0	0	0
ADD	1	0	0	1	1	0	00	0	0	0	0	0
SUB	1	0	0	1	1	0	00	1	0	0	0	0
AND	1	0	0	1	1	0	01	0	0	0	0	0
LD	1	1	0	0	1	0	10	0	0	0	0	0
ST	0	0	1	1	1	0	10	0	0	0	0	0
ADDI	1	0	0	1	0	1	00	0	0	0	0	0
BEQZ	0	0	0	1	0	0	00	0	1	0	0	0
BNEZ	0	0	0	1	0	0	00	0	0	1	0	0
JMP	0	0	0	0	0	0	00	0	0	0	1	0
SGEU	1	0	0	1	1	0	11	1	0	0	0	0
HALT	0	0	0	0	0	0	00	0	0	0	0	1

RW register write, MR load, MW store, UA/UB instruction reads ra/rb, YI ALU-Y = imm8, AOP ALU result select (00 adder, 01 AND, 10 pass-Y, 11 carry), SB subtract (invert Y, carry-in 1), BZ/BNZ/JP control transfer, HT halt. LD/ST use AOP=10 so the ALU passes the address register (rb) to EX/MEM; the store data is the forwarded ra operand.

6 Immediate and branch semantics

- LDI copies the 8-bit pattern; ADDI adds it modulo 256 (so ADDI R2,-3 = ADDI R2,0xFD).
- BEQZ/BNEZ test the (forwarded) value of ra in EX. JMP is unconditional. A taken transfer loads PC←imm8 at the end of EX and invalidates IF/ID and ID/EX: penalty = 2 cycles; not-taken branches cost nothing.
- HALT acts only when it is valid in EX: fetch stops, the two younger entries are invalidated, older instructions in MEM/WB finish. A HALT on a squashed path never reaches EX valid.

7 Application-specific instruction: SGEU Rd, Rs (opcode 0xC)

Semantics. Rd←1 if Rd≥Rs as *unsigned* 8-bit numbers, otherwise 0 (result is 0x00 or 0x01; Rs unchanged). R-format: 1100 ddd sss 000000.

Why. The controller must detect WATER_LEVEL≥target while filling and WATER_LEVEL≤threshold while draining, even if the sensor jumps over the threshold (e.g. 99→102 with target 100), so equality tests are useless. The base ISA has no flags; SUB alone cannot give an unsigned order when operands differ by more than 127 (e.g. 200 − 10 = 0xBE looks negative). A correct software routine must split on the operand MSBs:

```
MOV R2,R1; AND R2,R6                                  ; R6 = 0x80: MSB(WL)
MOV R3,R4; AND R3,R6                                  ; MSB(target)
SUB R3,R2; BEQZ R3,same                               ; equal MSBs?
BNEZ R2,ge; JMP lt                                    ; different MSBs: WL≥T iff MSB(WL)=1
same:  MOV R2,R1; SUB R2,R4; AND R2,R6; BEQZ R2,ge    ; same MSB: sign of the difference
```

That is 12 instructions, 3 extra registers, a mask constant and up to 3 taken branches (2 bubbles each) per sample. With SGEU the fill test is LD R1,[WL]; SGEU R1,R4; BEQZ R1,fill (3 instructions), and the drain test is LD R1,[WL]; MOV R2,R4; SGEU R2,R1; BEQZ R2,drain. The polling period falls from roughly 20–26 cycles to 6–7 cycles, so the valve or pump reacts sooner and uses fewer registers.

Datapath change. No new state. The ALU's 8-bit ripple adder already computes X + ~Y + 1 for SUB; its carry-out is 1 exactly when X≥$_u$Y. The carry is zero-extended ({0000000,CO}) and connected to input 3 of the ALU result 4:1 multiplexer. The decoder sets AOP=11 and SB=1 for opcode 0xC.

Pipeline timing and hazards. It is an ordinary one-cycle EX operation: both sources are read in ID (UA=UB=1), forwarded in EX like SUB, and the result is ready at the end of EX. It can therefore be forwarded from EX/MEM to the very next instruction (for example the following BEQZ) with no stall. If the instruction just before it is a LD of one of its sources, the normal load-use interlock inserts exactly one bubble. This happens in the fill loop, where LD R1,[WL] is immediately followed by SGEU R1,R4. SGEU causes no control hazard. It writes Rd only in WB, gated by the valid bit like every other write.